Mini Assignment: SSA, Dominators and Post Dominators

Question 1: Dominators and Post Dominators

CS3423 – Compilers II

November 2026

1 Dominators vs. post-dominators

A block A *dominates* block B if you can't reach B from the entry without passing through A first. A *post-dominator* is the same idea run backwards: A post-dominates B if every path from B to the exit goes through A. So dominators tell you what has definitely run *before* a block, and post-dominators tell you what will definitely run *after* it.

2 How the trees were made

Each program was compiled to LLVM IR with

```
clang++ -std=c++17 -S -emit-llvm -O0 -Xclang -disable-O0-optnone \
       -fno-discard-value-names N.cpp -o IR/N.ll
```

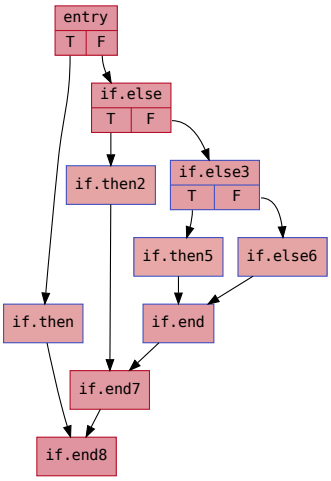
`-fno-discard-value-names` keeps readable block names like `if.then` and `for.cond` instead of `%5`. The trees were then printed with `opt -passes='print<domtree>,print<postdomtree>'` (output saved in `trees/N.txt`) and drawn with LLVM's `dot-cfg-only`, `dot-dom-only` and `dot-post-dom-only` passes. Running `./run.sh` redoes all of this.

A few notes for reading the tables:

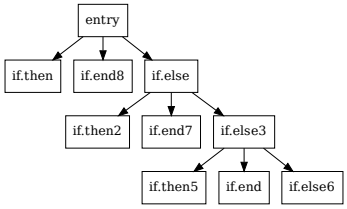
- Every block dominates and post-dominates itself, so the lists only show the *strict* relations.
- **idom** / **ipdom** are the immediate dominator and immediate post-dominator, i.e. the parent in each tree.
- LLVM puts a virtual `exit` node at the root of the post-dominator tree, which is why the function's last block has `exit` as its ipdom.
- Only the interesting function in each file is shown. `main` is a single block, so its trees are trivial.

3 Program 1: if / else-if / else ladder

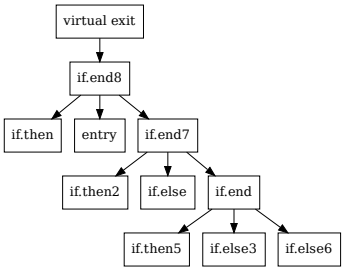
```
int classify(int x) {
    int result;
    if (x < 0) {
        result = -1;
    } else if (x == 0) {
        result = 0;
    } else {
        if (x % 2 == 0)
            result = 2; // positive even
        else
            result = 1; // positive odd
    }
    return result;
}
```



(a) CFG



(b) Dominator tree



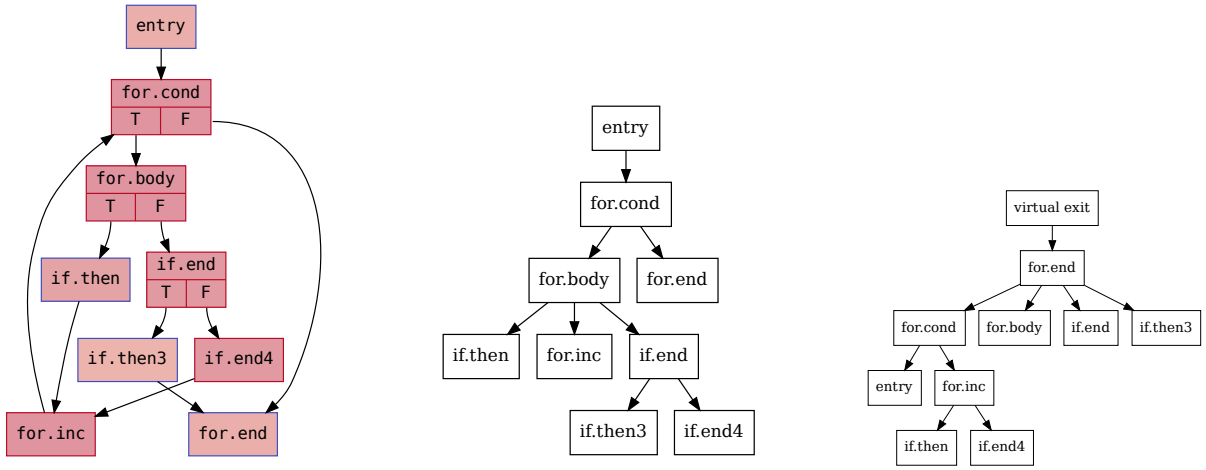
(c) Post-dominator tree

Block	idom	ipdom	Dominates	Post-dominates
`entry`	–	`if.end8`	all other blocks	–
`if.then`	`entry`	`if.end8`	–	–
`if.else`	`entry`	`if.end7`	`if.then2, if.else3, if.then5, if.else6, if.end, if.end7`	–
`if.then2`	`if.else`	`if.end7`	–	–
`if.else3`	`if.else`	`if.end`	`if.then5, if.else6, if.end`	–
`if.then5`	`if.else3`	`if.end`	–	–
`if.else6`	`if.else3`	`if.end`	–	–
`if.end`	`if.else3`	`if.end7`	–	`if.else3, if.then5, if.else6`
`if.end7`	`if.else`	`if.end8`	–	`if.else, if.then2, if.else3, if.then5, if.else6, if.end`
`if.end8`	`entry`	`exit`	–	all other blocks

What stands out: `if.end8` is the only way out, so it post-dominates every block. But its immediate dominator is `entry`, not `if.end7`, because the `x < 0` branch jumps straight to it. The nested if-else is what gives both trees their staircase shape.

4 Program 2: for loop with continue and break

```
int sumOdd(int n, int limit) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0)
            continue;
        sum += i;
        if (sum > limit)
            break;
    }
    return sum;
}
```



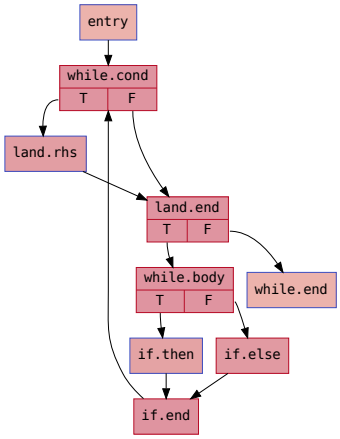
(a) CFG (b) Dominator tree (c) Post-dominator tree

Block	idom	ipdom	Dominates	Post-dominates
`entry`	–	`for.cond`	all other blocks	–
`for.cond`	`entry`	`for.end`	`for.body`, `if.then`, `if.end`, `if.then3`, `if.end4`, `for.inc`, `for.end`	`entry`, `if.then`, `if.end4`, `for.inc`
`for.body`	`for.cond`	`for.end`	`if.then`, `if.end`, `if.then3`, `if.end4`, `for.inc`	–
`if.then`	`for.body`	`for.inc`	–	–
`if.end`	`for.body`	`for.end`	`if.then3`, `if.end4`	–
`if.then3`	`if.end`	`for.end`	–	–
`if.end4`	`if.end`	`for.inc`	–	–
`for.inc`	`for.body`	`for.cond`	–	`if.then`, `if.end4`
`for.end`	`for.cond`	`exit`	–	all other blocks

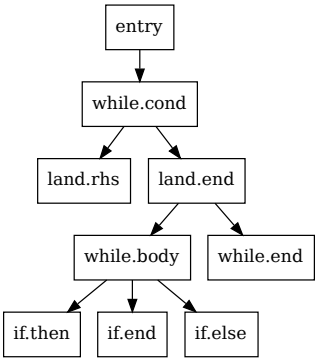
What stands out: the `break` changes things. Since `if.then3` jumps straight to `for.end`, the loop header `for.cond` no longer post-dominates `for.body` or `if.end`. The `continue` path (`if.then`) still goes through `for.inc`, so `for.inc` post-dominates it.

5 Program 3: while loop with && and an if-else

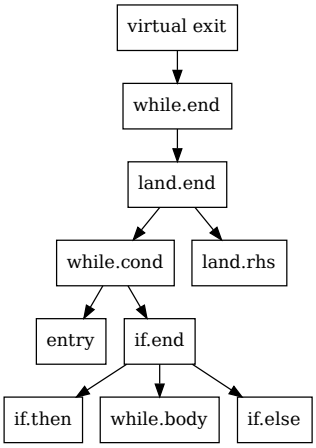
```
int collatzSteps(int n) {
    int steps = 0;
    while (n != 1 && steps < 1000) {
        if (n % 2 == 0)
            n = n / 2;
        else
            n = 3 * n + 1;
        steps++;
    }
    return steps;
}
```



(a) CFG



(b) Dominator tree



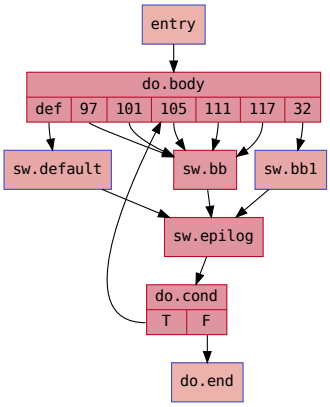
(c) Post-dominator tree

Block	idom	ipdom	Dominates	Post-dominates
`entry`	–	`while.cond`	all other blocks	–
`while.cond`	`entry`	`land.end`	`land.rhs`, `land.end`, `while.body`, `if.then`, `if.else`, `if.end`, `while.end`	`entry`, `while.body`, `if.then`, `if.else`, `if.end`
`land.rhs`	`while.cond`	`land.end`	–	–
`land.end`	`while.cond`	`while.end`	`while.body`, `if.then`, `if.else`, `if.end`, `while.end`	`entry`, `while.cond`, `land.rhs`, `while.body`, `if.then`, `if.else`, `if.end`
`while.body`	`land.end`	`if.end`	`if.then`, `if.else`, `if.end`	–
`if.then`	`while.body`	`if.end`	–	–
`if.else`	`while.body`	`if.end`	–	–
`if.end`	`while.body`	`while.cond`	–	`while.body`, `if.then`, `if.else`
`while.end`	`land.end`	`exit`	–	all other blocks

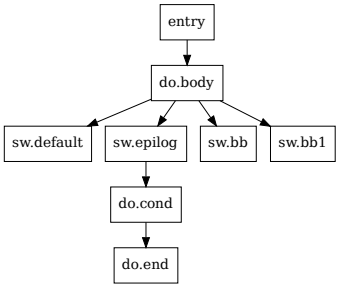
What stands out: the short-circuit `&&` splits the loop test into three blocks. `land.rhs` can be skipped (when `n == 1`), so it dominates nothing, while `land.end`, where both halves meet, post-dominates `while.cond`. The if-else in the body is the usual diamond: `while.body` dominates both arms and `if.end` post-dominates them.

6 Program 4: do-while with a switch

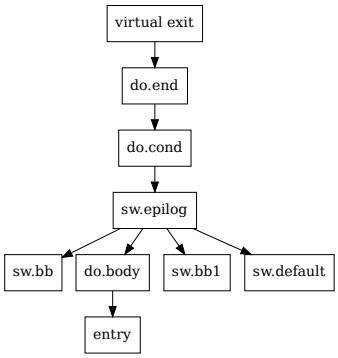
```
int wordScore(const char *s) {
    int score = 0;
    int i = 0;
    do {
        switch (s[i]) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            score += 2;
            break;
        case '␣':
            break;
        default:
            score += 1;
        }
        i++;
    } while (s[i] != '\0');
    return score;
}
```



(a) CFG



(b) Dominator tree



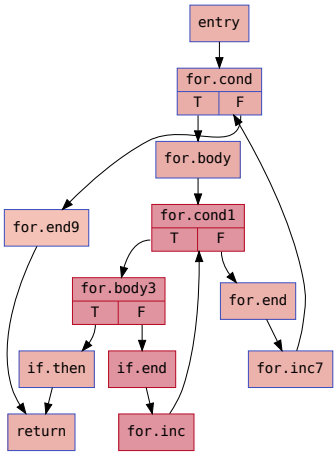
(c) Post-dominator tree

Block	idom	ipdom	Dominates	Post-dominates
`entry`	–	`do.body`	all other blocks	–
`do.body`	`entry`	`sw.epilog`	`sw.bb`, `sw.bb1`, `sw.default`, `sw.epilog`, `do.cond`, `do.end`	`entry`
`sw.bb`	`do.body`	`sw.epilog`	–	–
`sw.bb1`	`do.body`	`sw.epilog`	–	–
`sw.default`	`do.body`	`sw.epilog`	–	–
`sw.epilog`	`do.body`	`do.cond`	`do.cond`, `do.end`	`entry`, `do.body`, `sw.bb`, `sw.bb1`, `sw.default`
`do.cond`	`sw.epilog`	`do.end`	`do.end`	`entry`, `do.body`, `sw.bb`, `sw.bb1`, `sw.default`, `sw.epilog`
`do.end`	`do.cond`	`exit`	–	all other blocks

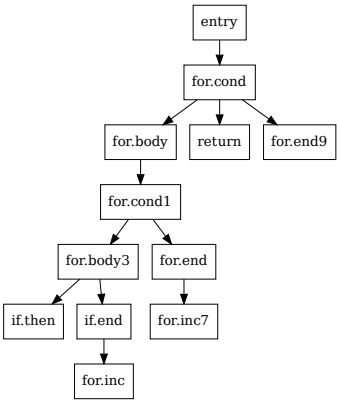
What stands out: a do-while always runs its body once, so `do.body`, `sw.epilog`, `do.cond` and `do.end` form a straight chain in the dominator tree. All three switch arms (the vowel case, the space case and default) meet at `sw.epilog`, which makes it their immediate post-dominator.

7 Program 5: nested for loops with an early return

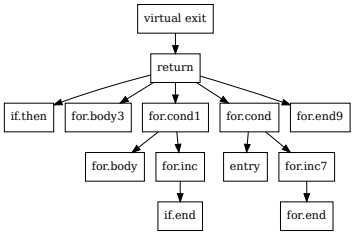
```
bool hasDuplicate(int a[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j])
                return true;
        }
    }
    return false;
}
```



(a) CFG



(b) Dominator tree



(c) Post-dominator tree

Block	idom	ipdom	Dominates	Post-dominates
`entry`	–	`for.cond`	all other blocks	–
`for.cond`	`entry`	`return`	`for.body`, `for.cond1`, `for.body3`, `if.then`, `if.end`, `for.inc`, `for.end`, `for.inc7`, `for.end9`, `return`	`entry`, `for.end`, `for.inc7`
`for.body`	`for.cond`	`for.cond1`	`for.cond1`, `for.body3`, `if.then`, `if.end`, `for.inc`, `for.end`, `for.inc7`	–
`for.cond1`	`for.body`	`return`	`for.body3`, `if.then`, `if.end`, `for.inc`, `for.end`, `for.inc7`	`for.body`, `if.end`, `for.inc`
`for.body3`	`for.cond1`	`return`	`if.then`, `if.end`, `for.inc`	–
`if.then`	`for.body3`	`return`	–	–
`if.end`	`for.body3`	`for.inc`	`for.inc`	–
`for.inc`	`if.end`	`for.cond1`	–	`if.end`
`for.end`	`for.cond1`	`for.inc7`	`for.inc7`	–
`for.inc7`	`for.end`	`for.cond`	–	`for.end`
`for.end9`	`for.cond`	`return`	–	–
`return`	`for.cond`	`exit`	–	all other blocks

What stands out: the early `return true`. Normally a loop's exit block post-dominates its header, but here `for.end` does not post-dominate `for.cond1`, because you can leave the inner loop through `if.then` and go straight to `return`. So the ipdom of `for.cond1` is `return` itself. Also, `return` is reached from both `if.then` and `for.end9`, so its immediate dominator is the outer loop header `for.cond`.